Predicting Medical Costs

Sri Manju Baargavi Rangaraj Ganesh (48780901)

Introduction

This report focuses on predicting annual medical claim expenses for policyholders using demographic, lifestyle, and wearable device data. Accurate prediction of healthcare costs is essential for insurers to set fair premiums, manage financial risk, and design targeted wellness programs.

The analysis begins with data preprocessing and exploratory data analysis (EDA) to understand key patterns and relationships in the dataset. This is followed by the development and evaluation of multiple predictive models, including regularised regression, K-Nearest Neighbours (KNN), and Random Forest.

The objective is to identify the most effective model for predicting medical expenses and to determine the key factors that influence healthcare costs.

Part (a): Data Loading

```
#Loading required library
library(tidyverse)
#Loading dataset
df <- read.csv("healthinsurance.csv")
str(df)
```

```
'data.frame':   25000 obs. of  11 variables:
 $ age                  : int  58 50 26 34 50 19 45 67 32 51 ...
 $ region               : chr  "metro" "regional" "regional" "metro" ...
 $ bmi                  : num  26.6 26.9 30.6 23.3 27.2 29 17.4 21.5 33.4 27.1 ...
 $ plan_type            : chr  "basic" "silver" "basic" "gold" ...
 $ daily_steps          : int  7503 4690 4668 8710 6784 7340 10253 10037 8279 6243 ...
 $ smoker_ind           : int  1 0 0 0 0 0 0 0 0 1 ...
```

```
 $ chronic_condition_ind: int  0 0 0 0 1 0 0 1 1 0 ...
 $ sleep_hours_ind      : int  0 0 0 1 1 0 0 0 0 0 ...
 $ high_activity_ind    : int  1 0 0 1 1 0 1 1 0 0 ...
 $ prev_claims_ind      : int  0 0 1 0 1 0 0 1 1 0 ...
 $ expenses             : num  0 0 615 0 1376 ...
```

The dataset was loaded using read.csv() and its structure was examined using str(). It contains 25,000 observations and 11 variables, including numerical variables (age, bmi, daily_steps, expenses), categorical variables (region, plan_type), and binary indicator variables stored as integers. The response variable expenses contains a large proportion of zero values, indicating a highly skewed distribution.

```
#Converting to factors
df$region <- as.factor(df$region)

df$plan_type <- factor(df$plan_type,
                       levels = c("basic","bronze","silver","gold"))
#Checking levels
levels(df$region)
```

```
[1] "metro"    "regional" "remote"
```

```
levels(df$plan_type)
```

```
[1] "basic"  "bronze" "silver" "gold"
```

The variables region and plan_type were converted to factors to ensure correct handling in modelling. The variable plan_type was defined as an ordered factor (basic, bronze, silver, gold) to reflect the hierarchy of insurance plans. Factor levels were reviewed to confirm they are correctly specified.

```
df <- df %>% filter(expenses > 0)
str(df)
```

```
'data.frame':   4910 obs. of  11 variables:
 $ age                  : int  26 50 32 18 60 71 69 45 59 69 ...
 $ region               : Factor w/ 3 levels "metro","regional",..: 2 1 2 2 1 1 1 1 1 2 ...
 $ bmi                  : num  30.6 27.2 33.4 21.3 32.1 33 16 26.4 27.7 24.6 ...
 $ plan_type            : Factor w/ 4 levels "basic","bronze",..: 1 3 2 3 3 2 3 2 3 3 ...
 $ daily_steps          : int  4668 6784 8279 6866 5660 4662 2778 7884 9276 4036 ...
```

```
$ smoker_ind           : int  0 0 0 1 1 1 0 0 1 0 ...
$ chronic_condition_ind: int  0 1 1 0 0 0 1 1 0 0 ...
$ sleep_hours_ind      : int  0 1 0 0 0 0 0 0 0 0 ...
$ high_activity_ind    : int  0 1 0 0 0 0 0 0 0 0 ...
$ prev_claims_ind      : int  1 1 1 0 0 0 0 1 1 0 ...
$ expenses             : num  615 1376 2762 520 1309 ...
```

Observations with zero expenses were removed to retain only positive claim amounts. This is necessary as the analysis focuses on modelling claim severity, and continuous models require strictly positive response values. After filtering, the dataset contains 4,910 observations with correctly formatted variables, making it suitable for further analysis and modelling.

Part (b): Data Splitting

```r
#Setting a seed using my MQ OneID
set.seed(48780901)
library(rsample)
#Creating  an 80-20 train-test split with stratification on expenses
split <- initial_split(df, prop = 0.8, strata = expenses)
#Extracting training data (80%)
train_data <- training(split)
#Extracting testing data (20%)
test_data  <- testing(split)
```

The dataset was split into training (80%) and testing (20%) sets using stratified sampling based on expenses. Stratification is important because the response variable is highly right-skewed, with many small values and fewer large values. By preserving the distribution of expenses in both datasets, stratification ensures that the training and test sets are representative of the overall data, leading to more reliable model training and evaluation.

```r
#Creating 10-fold cross-validation object using training data
cv_folds <- vfold_cv(train_data, v = 10)
```

A 10-fold cross-validation object was created using the training dataset. This divides the training data into 10 folds, where each fold is used once as a validation set while the remaining folds are used for training.

Part (c): Exploratory Data Analysis (EDA)

```
colSums(is.na(train_data))
```

```
                  age                region                   bmi
                    0                     0                     0
            plan_type           daily_steps           smoker_ind
                    0                     0                     0
chronic_condition_ind       sleep_hours_ind     high_activity_ind
                    0                     0                     0
      prev_claims_ind              expenses
                    0                     0
```

The training dataset was checked for missing values using colSums(is.na()). No missing values were detected across the variables. Therefore, no imputation or data cleaning for missing values is required.

```
table(train_data$region)
```

```
   metro regional   remote
    2081     1403      442
```

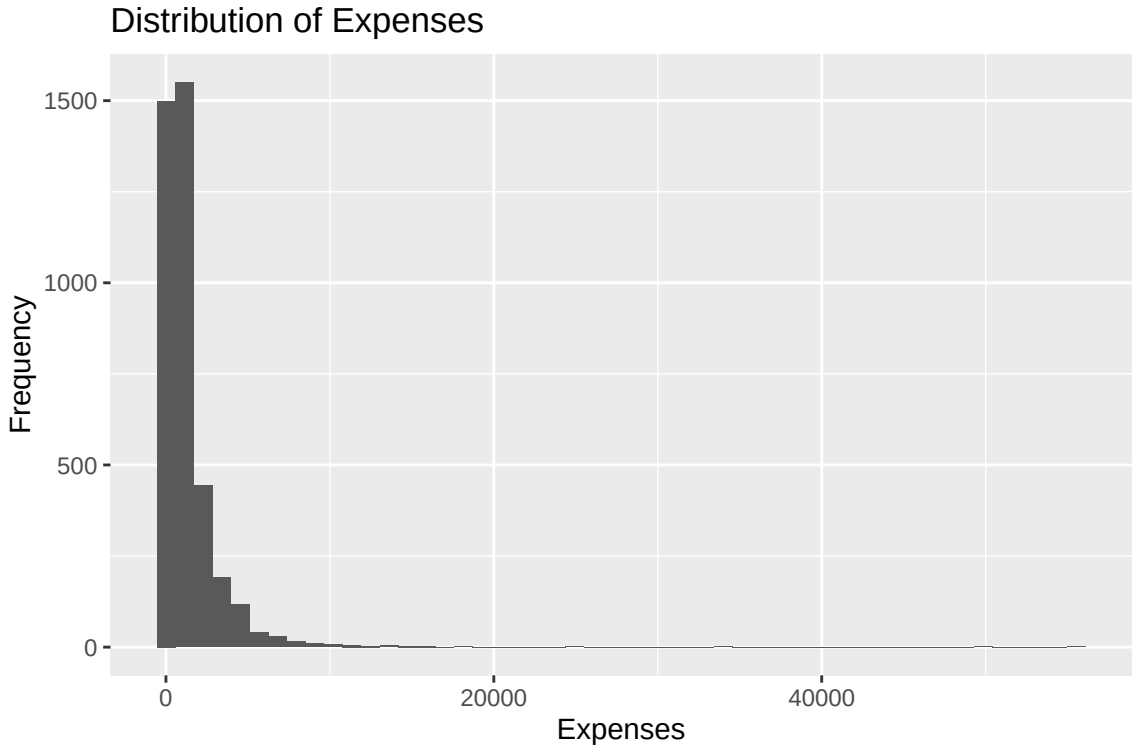
```
table(train_data$plan_type)
```

```
 basic bronze silver   gold
  1146   1336   1065    379
```

The frequency distribution of the categorical variables was examined using contingency tables. While some categories such as remote (region) and gold (plan_type) have fewer observations compared to others, they still represent a meaningful proportion of the dataset.

Therefore, no factor levels are considered extremely rare, and there is no need to merge or collapse categories. Retaining all levels preserves important information and interpretability for modelling.

```r
library(ggplot2)
#Plotting histogram to examine the distribution of the response variable (expenses)
ggplot(train_data, aes(x = expenses)) +
  geom_histogram(bins = 50) + # Use 50 bins to get a detailed view of distribution
  labs(title = "Distribution of Expenses",
       x = "Expenses",
       y = "Frequency")
```



The histogram of expenses shows a highly right-skewed distribution, with a large concentration of observations at lower expense values and a long tail extending towards higher values. This indicates that most policyholders incur relatively low medical costs, while a smaller number generate substantially higher expenses.

Due to this skewness, a transformation such as a logarithmic transformation may be beneficial, particularly for models like regularised regression, which assume a more symmetric distribution and can be sensitive to extreme values.

However, transformation may not be necessary for models such as K-Nearest Neighbours (KNN) and Random Forest, as these models are less sensitive to skewness and do not rely on strict distributional assumptions.

```r
#Selecting numerical variables for analysis
num_vars <- train_data %>%
  select(age, bmi, daily_steps, expenses)
#Computing correlation matrix
cor(num_vars)
```

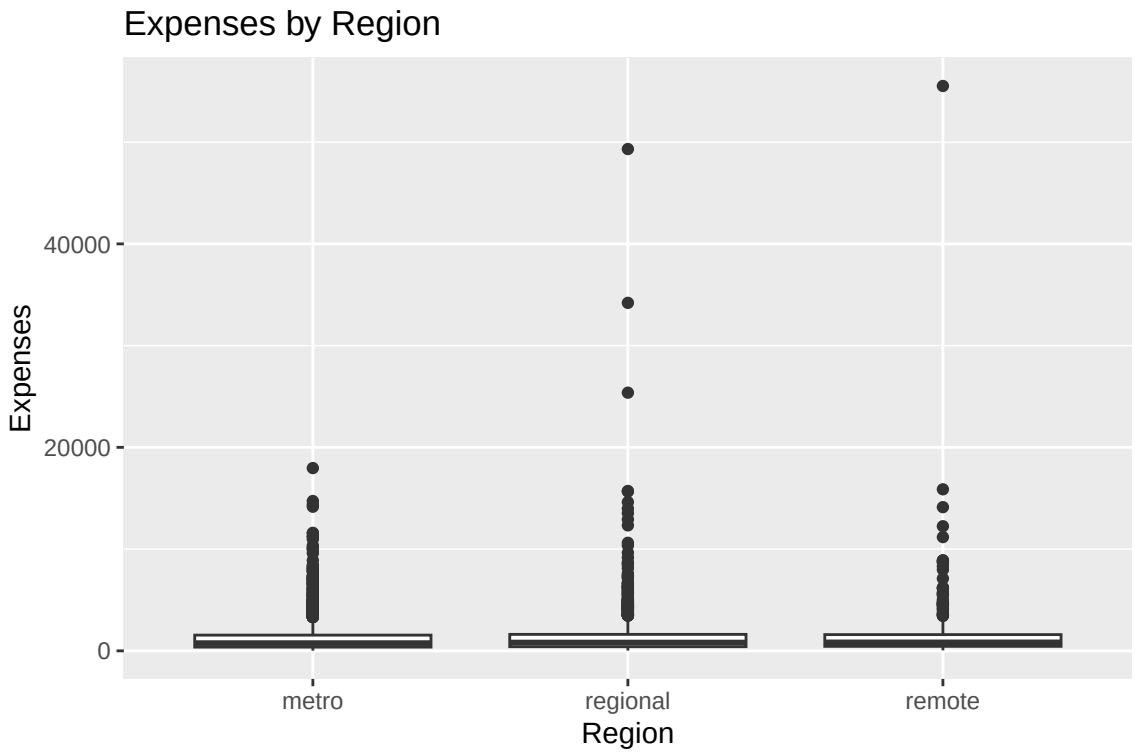
```
                  age         bmi daily_steps    expenses
age          1.00000000 -0.02739640 -0.00679714  0.14103275
bmi         -0.02739640  1.00000000  0.01256313  0.10718218
daily_steps -0.00679714  0.01256313  1.00000000 -0.04462371
expenses     0.14103275  0.10718218 -0.04462371  1.00000000
```

The relationships among numerical variables were examined using a correlation matrix. The correlations between the predictor variables (age, bmi, and daily_steps) are all very weak, indicating no evidence of multicollinearity.

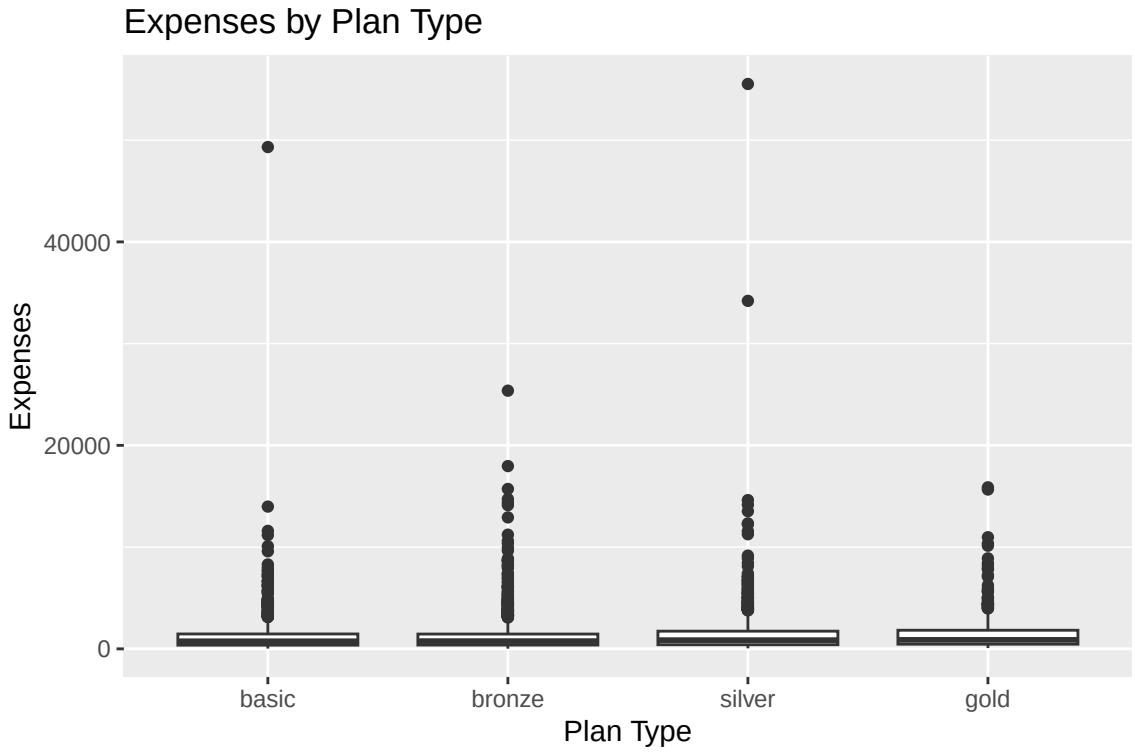
The response variable expenses shows a weak positive correlation with age (0.14) and bmi (0.11), suggesting that higher age and BMI are associated with increased medical costs. In contrast, daily_steps has a weak negative correlation with expenses (-0.04), indicating a minimal relationship.

Overall, age and bmi appear to be more promising predictors of medical expenses, while daily_steps is likely to have a limited impact in the predictive models.

```r
#Relationship between expenses and region
ggplot(train_data, aes(x = region, y = expenses)) +
  geom_boxplot() +   # Boxplot to compare distributions across regions
  labs(title = "Expenses by Region",
       x = "Region",
       y = "Expenses")
```




```
#Relationship between expenses and plan_type
ggplot(train_data, aes(x = plan_type, y = expenses)) +
  geom_boxplot() +   # Boxplot to compare distributions across plan types
  labs(title = "Expenses by Plan Type",
       x = "Plan Type",
       y = "Expenses")
```



The relationship between the response variable expenses and categorical predictors was examined using boxplots. The distributions of expenses vary across both region and plan_type, indicating that these variables influence medical costs.

For region, the distributions are relatively similar, although remote and regional areas show slightly higher variability and extreme values. This suggests that region may have some effect, but it is not a strong differentiator.

In contrast, plan_type shows clearer differences, with higher-tier plans such as silver and gold exhibiting greater variability and higher expense values. This indicates that plan_type is a more influential predictor of medical costs.

Overall, plan_type emerges as a particularly promising categorical predictor, while region may still provide some additional predictive value.

Part (d): Model Development and Evaluation

```
#Loading necessary libraries
library(tidyverse)
library(tidymodels)
library(glmnet)
library(kknn)
library(ranger)
#Setting RMSE as the main evaluation metric
rmse_metric <- metric_set(rmse)
```

Regularised Regression Model

```
#Defining regularised regression model
reg_model <- linear_reg(
  penalty = tune(),    # controls strength of regularisation
  mixture = tune()     # controls L1 (lasso) vs L2 (ridge)
) %>%
  set_engine("glmnet")
```

A regularised regression model was specified using the glmnet engine. The penalty parameter controls the strength of regularisation, while the mixture parameter determines the balance between L1 (lasso) and L2 (ridge) penalties.

```
#Preprocessing steps for regression model
reg_recipe <- recipe(expenses ~ ., data = train_data) %>%
  step_dummy(all_nominal_predictors()) %>% #Converting categorical variables to dummy variabl
  step_zv(all_predictors()) %>% #Removing zero variance predictors
  step_normalize(all_numeric_predictors())  #Standardising predictors
```

A preprocessing recipe was created to prepare the data for modelling. Categorical variables were converted into dummy variables, zero-variance predictors were removed, and numerical predictors were standardised. Normalisation is particularly important for regularised regression as it ensures all predictors are on a comparable scale.

```
#Combining model and preprocessing into workflow
reg_workflow <- workflow() %>%
  add_recipe(reg_recipe) %>%
  add_model(reg_model)
```

A workflow was constructed by combining the preprocessing recipe and the model specification. This ensures that all preprocessing steps are consistently applied during model training and evaluation.

```
#Creating grid with at least 50 combinations
reg_grid <- grid_regular(
  penalty(range = c(-4, 1)),   #10^-4 to 10^1
  mixture(),
  levels = c(10, 6)            #60 combinations
)
#Tuning model using 10-fold cross-validation
reg_tuned <- tune_grid(
  reg_workflow,
  resamples = cv_folds,
  grid = reg_grid,
  metrics = metric_set(rmse)
)
```

Hyperparameter tuning was performed using a grid search over a range of penalty and mixture values. A total of 60 combinations were evaluated using 10-fold cross-validation, and model performance was assessed using RMSE.

```
#Selecting best hyperparameters
best_reg <- select_best(reg_tuned, metric = "rmse")
best_reg
```

```
# A tibble: 1 x 3
  penalty mixture .config
    <dbl>   <dbl> <chr>
1  0.0001       0 pre0_mod01_post0
```

For the regularised regression model, the best hyperparameter combination was a penalty value of 0.0001 and a mixture value of 0. Since the mixture parameter equals 0, the selected model corresponds to ridge regression rather than lasso or elastic net. The small penalty value indicates that only a weak degree of regularisation was required to achieve the best predictive performance.

```
#Finalising workflow with best hyperparameters
final_reg_workflow <- finalize_workflow(reg_workflow, best_reg)
```

K-Nearest Neighbours (KNN) Model

```
#Defining KNN regression model with tunable hyperparameters
knn_model <- nearest_neighbor(
  neighbors = tune(),     # number of neighbours
  dist_power = tune(),    # distance metric (e.g., Euclidean)
  weight_func = tune()    # weighting scheme
) %>%
  set_engine("kknn") %>%
  set_mode("regression")
```

A K-Nearest Neighbours (KNN) regression model was specified using the kknn engine. The model includes three tuning parameters: neighbors, which determines the number of nearest neighbours; dist_power, which controls the distance metric; and weight_func, which specifies how neighbours are weighted.

```
#Preprocessing for KNN model
knn_recipe <- recipe(expenses ~ ., data = train_data) %>%
  step_dummy(all_nominal_predictors()) %>%    # Convert categorical variables to dummy varial
  step_zv(all_predictors()) %>%               # Remove zero variance predictors
  step_normalize(all_numeric_predictors())   # Standardise predictors (VERY important for KN
```

A preprocessing recipe was created for the KNN model. Categorical variables were converted into dummy variables, zero-variance predictors were removed, and numerical predictors were standardised. Normalisation is essential for KNN as it is a distance-based method and is sensitive to the scale of predictors.

```
#Combining model and preprocessing into workflow
knn_workflow <- workflow() %>%
  add_recipe(knn_recipe) %>%
  add_model(knn_model)
```

A workflow was constructed by combining the preprocessing steps and the KNN model specification. This ensures consistent application of preprocessing during model training and evaluation.

```
#Creating tuning grid (at least 50 combinations)
knn_grid <- grid_regular(
  neighbors(range = c(1L, 25L)),
  dist_power(range = c(1, 2)),
```

```
  weight_func(values = c("rectangular", "triangular", "inv")),
  levels = c(10, 2, 3)   # 10 × 2 × 3 = 60 combinations
)
#Tuning model using 10-fold cross-validation
knn_tuned <- tune_grid(
  knn_workflow,
  resamples = cv_folds,
  grid = knn_grid,
  metrics = metric_set(rmse)
)
```

Hyperparameter tuning was conducted using a grid search over values of neighbors, dist_power, and weight_func. A total of 60 combinations were evaluated using 10-fold cross-validation, resulting in 600 models being trained. Model performance was assessed using RMSE.

```
#Selecting best KNN model
best_knn <- select_best(knn_tuned, metric = "rmse")
best_knn
```

```
# A tibble: 1 x 4
  neighbors weight_func dist_power .config
      <int> <chr>            <dbl> <chr>
1        25 inv                  1 pre0_mod55_post0
```

For the KNN model, the best hyperparameter combination consisted of 25 neighbours, an inverse distance weighting function, and a distance power of 1. This indicates that the best-performing KNN model used a relatively large neighbourhood, gave greater importance to closer observations, and relied on Manhattan distance to measure similarity between cases.

```
#Finalising workflow with best KNN hyperparameters
final_knn_workflow <- finalize_workflow(knn_workflow, best_knn)
```

Random Forest Model

```
#Defining Random Forest regression model with tunable hyperparameters
rf_model <- rand_forest(
  mtry = tune(),     # number of predictors sampled at each split
  min_n = tune(),    # minimum observations required to split a node
```

```
  trees = tune()     # total number of trees
) %>%
  set_engine("ranger") %>%
  set_mode("regression")
```

A Random Forest regression model was specified using the ranger engine. The model includes three tuning parameters: mtry, which determines the number of predictors sampled at each split; min_n, which controls the minimum number of observations required to split a node; and trees, which specifies the total number of trees in the forest.

```
#Preprocessing for Random Forest model
rf_recipe <- recipe(expenses ~ ., data = train_data) %>%
  step_dummy(all_nominal_predictors()) %>%   # Convert categorical variables to dummy variabl
  step_zv(all_predictors())      # Remove zero variance predictors
```

A preprocessing recipe was created for the Random Forest model. Categorical variables were converted into dummy variables, and zero-variance predictors were removed. Unlike KNN and regularised regression, normalisation is not required for Random Forest because tree-based models are not sensitive to the scale of predictors.

```
#Combining model and preprocessing into workflow
rf_workflow <- workflow() %>%
  add_recipe(rf_recipe) %>%
  add_model(rf_model)
```

A workflow was constructed by combining the preprocessing recipe and the Random Forest model specification. This ensures that preprocessing is applied consistently across all resamples during model training and evaluation.

```
#Preparing processed training data to finalise the mtry range
rf_prep <- prep(rf_recipe)
rf_train_processed <- bake(rf_prep, new_data = NULL)
#Creating tuning grid with exactly 50 combinations
rf_grid <- grid_regular(
  finalize(mtry(), rf_train_processed %>% select(-expenses)),
  min_n(range = c(2L, 10L)),
  trees(range = c(50L, 150L)),
  levels = c(5, 5, 2)
)
library(parallel)
library(doParallel)
```

```
cl <- makeCluster(parallel::detectCores() - 1)
registerDoParallel(cl)
#Tuning Random Forest model using 10-fold cross-validation
rf_tuned <- tune_grid(
  rf_workflow,
  resamples = cv_folds,
  grid = rf_grid,
  metrics = metric_set(rmse)
)
stopCluster(cl)
```

Hyperparameter tuning was conducted using a grid search over values of mtry, min_n, and trees. A total of 50 combinations were evaluated using 10-fold cross-validation, resulting in 500 models being trained. Model performance was assessed using RMSE.

```
#Selecting best Random Forest model
best_rf <- select_best(rf_tuned, metric = "rmse")
best_rf
```

```
# A tibble: 1 x 4
   mtry trees min_n .config
  <int> <int> <int> <chr>
1     4   150    10 pre0_mod20_post0
```

For the Random Forest model, the best hyperparameter combination was mtry = 4, trees = 150, and min_n = 10. This indicates that the best-performing model considered four predictors at each split, used a forest of 150 trees, and required at least 10 observations before a node could be split. These settings suggest a balance between model flexibility and control of overfitting.

```
#Finalising workflow with best RF hyperparameters
final_rf_workflow  <- finalize_workflow(rf_workflow, best_rf)
```

Hyperparameter tuning

Hyperparameter tuning for each model was performed using 10-fold cross-validation on the training dataset. The data was divided into 10 approximately equal folds. In each iteration, 9 folds were used to train the model, while the remaining fold was used as a validation set. This process was repeated 10 times so that each fold served as the validation set once, and the average RMSE across all folds was used to evaluate model performance.

Grid search was used to systematically explore different combinations of hyperparameters for each model. For each workflow, 50 hyperparameter combinations were evaluated. Since each combination was assessed across 10 folds, this resulted in a total of 500 fitted models per workflow. The combination that achieved the lowest average RMSE across the folds was selected as the best set of hyperparameters.

Model Ranking

```
#Regularised Regression
reg_results <- collect_metrics(reg_tuned) %>%
  filter(.metric == "rmse") %>%
  arrange(mean) %>%
  slice(1) %>%
  mutate(model = "Regularised Regression")
#KNN
knn_results <- collect_metrics(knn_tuned) %>%
  filter(.metric == "rmse") %>%
  arrange(mean) %>%
  slice(1) %>%
  mutate(model = "KNN")
#Random Forest
rf_results <- collect_metrics(rf_tuned) %>%
  filter(.metric == "rmse") %>%
  arrange(mean) %>%
  slice(1) %>%
  mutate(model = "Random Forest")
#Combine and rank
model_ranking <- bind_rows(reg_results, knn_results, rf_results) %>%
  select(model, mean, std_err) %>%
  arrange(mean)
model_ranking
```

```
# A tibble: 3 x 3
  model                  mean std_err
  <chr>                 <dbl>   <dbl>
1 Regularised Regression 1786.    216.
2 Random Forest          1789.    206.
3 KNN                    1835.    214.
```

The best hyperparameter combination for each model was selected based on the lowest cross-validated RMSE. The models were then ranked according to their RMSE values.

The regularised regression model achieved the lowest RMSE of 1786.40, followed closely by the Random Forest model with an RMSE of 1789.25. The KNN model performed comparatively worse, with a higher RMSE of 1835.17.

Overall, the regularised regression model performed the best, as it produced the lowest prediction error. The very similar performance of Random Forest suggests that there are limited non-linear relationships in the data, and simpler linear models are sufficient to capture the underlying patterns. In contrast, the KNN model showed weaker performance, likely due to its sensitivity to noise and the relatively weak relationships among predictors identified during the exploratory data analysis.

Part (e): Model Performance and Predictor Importance

1.

```
#Fitting final model on full training data
final_reg_fit <- fit(final_reg_workflow, data = train_data)
#Generating predictions on test set
test_predictions <- predict(final_reg_fit, new_data = test_data) %>%
  bind_cols(test_data)
#Calculating RMSE on test set
test_rmse <- rmse(test_predictions, truth = expenses, estimate = .pred)
test_rmse
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard       1438.
```

The final regularised regression model was evaluated on the test dataset. The test RMSE was 1437.93, compared to a cross-validated RMSE of 1786.40 obtained during model tuning.

The test RMSE is lower than the cross-validated RMSE, indicating that the model generalises well to unseen data. This suggests that the cross-validation process provided a slightly conservative estimate of model performance, and there is no evidence of overfitting. Overall, the model demonstrates strong predictive capability on new data.

2.

```
#Extracting model coefficients
library(broom)
reg_coefs <- tidy(final_reg_fit) %>%
  filter(term != "(Intercept)") %>%
  arrange(desc(abs(estimate)))
reg_coefs
```

```
# A tibble: 13 x 3
   term                  estimate penalty
   <chr>                    <dbl>   <dbl>
 1 chronic_condition_ind    687.   0.0001
 2 smoker_ind               376.   0.0001
 3 age                      335.   0.0001
 4 bmi                      296.   0.0001
 5 prev_claims_ind          292.   0.0001
 6 high_activity_ind       -112.   0.0001
 7 sleep_hours_ind          109.   0.0001
 8 plan_type_silver         106.   0.0001
 9 plan_type_gold           100.   0.0001
10 daily_steps              -89.1  0.0001
11 region_remote             64.6  0.0001
12 region_regional           32.0  0.0001
13 plan_type_bronze          17.8  0.0001
```

The most important predictors were identified based on the magnitude of the coefficients from the final regularised regression model. Variables with larger absolute coefficient values have a greater influence on predicted medical expenses.

The most influential predictors include chronic_condition_ind, smoker_ind, age, bmi, and prev_claims_ind. These variables have the largest coefficients in magnitude, indicating that they play a significant role in determining medical costs. In particular, the presence of chronic conditions and smoking status are associated with substantial increases in predicted expenses, while age and BMI also contribute positively to higher costs.

3.

The key predictors identified from the final model largely align with the findings from the exploratory data analysis. In Part (c), age and bmi showed weak positive relationships with

medical expenses, and both variables were also identified as important predictors in the final model, confirming their influence on healthcare costs.

However, some predictors such as chronic_condition_ind and smoker_ind were not strongly highlighted in the earlier numerical analysis but emerged as highly influential in the model. This suggests that while their linear correlations with expenses may be modest, they have a substantial impact when considered within a multivariate modelling framework.

Overall, there are no major contradictions between the EDA and modelling results. The model builds upon and refines the initial observations, highlighting additional important predictors that were not as evident in the preliminary analysis.

Part (f): Reflecting on Predictive Modelling Journey

During the implementation of the Random Forest model, a key challenge encountered was excessive computation time during hyperparameter tuning. The model initially took more than 10–15 minutes to run, which significantly slowed down the workflow.

The warning messages included issues related to parallel processing, such as "closing unused connection" and delays during the execution of the tuning process. This indicated that the model was computationally expensive due to the large number of trees and hyperparameter combinations being evaluated.

To debug the issue, the R session was restarted to clear any existing parallel connections, and the tuning grid was simplified by reducing the range of the trees parameter and limiting the total number of combinations to 50. Additionally, parallel processing was implemented using the doParallel package to distribute computations across multiple CPU cores.

These steps significantly reduced the runtime while still meeting the assignment requirements. This experience highlighted the importance of balancing model complexity with computational efficiency and reinforced the need to carefully manage resources when working with resampling and grid search in tidymodels.
